

Running Notes-29. LangChain & Agentic AI – From Basics to Advanced 29

Messages and Chat Models

MCQ 1 of 25

Which message type is mainly used to give **high-level instructions or context** to a chat model?

- A) HumanMessage
- B) AIMessage
- C) SystemMessage
- D) ResponseMessage

👉 Your answer?

✅ **Correct — C) SystemMessage**

One-line explanation: SystemMessage gives high-level instructions or context that guides how the model should behave.

Learning conclusion: SystemMessage = Instructions / Context for the model.

MCQ 2 of 25

Which message type represents the **user's input** in a conversation?

- A) SystemMessage
- B) HumanMessage
- C) AIMessage
- D) ModelMessage

 **Your answer?**

 **Correct — B) HumanMessage**

One-line explanation: HumanMessage represents the message or input coming from the user.

Learning conclusion: HumanMessage = User's question or request.

MCQ 3 of 25

What does a LangChain chat model usually return when we call `invoke()` with messages?

- A) Plain string
- B) Dictionary
- C) AIMessage
- D) HumanMessage

 **Your answer?**

✅ **Correct — C) AIMessage**

One-line explanation: When a LangChain chat model is invoked with messages, it typically returns an AIMessage object, not just a plain string.

Learning conclusion: `invoke()` → AIMessage → use `.content` to get the actual answer text.

MCQ 4 of 25

How do we access only the **main text content** from a returned AI message?

- A) `response.text`
- B) `response.message`
- C) `response.content`
- D) `response.output`

👉 **Your answer?**

✅ **Correct — C) response.content**

One-line explanation: `response` is the complete AI message object, while `response.content` gives only the actual answer text.

Learning conclusion: Message object = complete response; `.content` = actual text.

MCQ 5 of 25

Why are **message roles** important in a chat application?

- A) They reduce Python code size
- B) They tell who said what and the purpose of the message
- C) They automatically save conversation permanently
- D) They increase internet speed

👉 Your answer?

✅ **Correct — B) They tell who said what and the purpose of the message**

One-line explanation: Roles give structure to the conversation by clearly separating **System → instruction, Human → question, AI → response**.

Learning conclusion: Role tells **WHO** is speaking and what purpose that message has.

MCQ 6 of 25

In the following code, what is messages?

```
messages = [  
    SystemMessage(content="You are a teacher."),  
    HumanMessage(content="What is Python?")  
]
```

- A) A string
- B) A tuple
- C) A Python list containing message objects
- D) An AIMessage

👉 Your answer?

✅ **Correct — C) A Python list containing message objects**

One-line explanation: messages is a normal Python list that stores objects like SystemMessage and HumanMessage, and this complete list can be sent to the chat model.

Learning conclusion: Multiple chat messages are usually grouped inside a Python list.

MCQ 7 of 25

Why does the **order of messages** matter in a conversation?

- A) Because Python lists require alphabetical order
- B) Because the sequence represents the conversation/context given to the model
- C) Because SystemMessage must always be last
- D) Because order affects API key security

👉 Your answer?

✅ **Correct — B) Because the sequence represents the conversation/context given to the model**

One-line explanation: Message order matters because the list represents an **ordered conversation**, such as system instruction first and human request next.

Learning conclusion: Messages are not a random collection; their order gives conversation structure.

MCQ 8 of 25

In a basic independent `llm.invoke()` call, does the model automatically remember all previous calls?

- A) Yes, always
- B) Only if the API key is same
- C) No, previous context must be supplied or managed
- D) Only for SystemMessage

 **Your answer?**

 **Correct — C) No, previous context must be supplied or managed**

One-line explanation: In basic independent calls, the model does not automatically carry previous conversation state; the application must provide or manage that history.

Learning conclusion: Basic model calls are stateless unless conversation context is supplied again.

MCQ 9 of 25

What does **stateless** mean in this context?

- A) The model has no output
- B) The current call does not automatically carry previous conversation state
- C) The model cannot accept multiple messages
- D) The model cannot use a SystemMessage

👉 Your answer?

✅ **Correct — B) The current call does not automatically carry previous conversation state**

One-line explanation: Stateless means each basic call is independent unless previous conversation context is again provided or managed by the application.

Learning conclusion: Stateless = no automatic memory between independent calls.

MCQ 10 of 25

Why do we send previous messages again in a multi-turn conversation?

- A) To make Python code longer
- B) To give the model the context needed for follow-up questions

- C) To change the API key
- D) To convert AIMessage into HumanMessage

👉 **Your answer?**

✅ **Correct — B) To give the model the context needed for follow-up questions**

One-line explanation: Previous messages help the model understand follow-up questions whose meaning depends on earlier conversation.

Learning conclusion: Conversation history gives useful context to understand the current question.

MCQ 11 of 25

In a simple LangChain chatbot, who usually maintains the messages history list?

- A) The model itself
- B) OpenAI server automatically
- C) Our Python application
- D) SystemMessage

👉 **Your answer?**

✅ **Correct — C) Our Python application**

One-line explanation: In the simple chatbot shown in the document, our Python application keeps the messages list,

sends it to the model, and stores the AI response back into that history.

Learning conclusion: The application maintains conversation history; the model does not magically maintain our Python list.

MCQ 12 of 25

What happens if we close the simple Python chatbot program where history is stored only in a messages list?

- A) The history is saved permanently
- B) The history is automatically stored by the model
- C) The in-memory history disappears
- D) The API key stores the history

 **Your answer?**

 **Correct — C) The in-memory history disappears**

One-line explanation: In the simple chatbot, the messages list exists only while the Python program is running; after closing the program, that history is lost.

Learning conclusion: Python list history is temporary memory, not persistent memory.

MCQ 13 of 25

Why can't we keep sending unlimited conversation history forever?

- A) Python lists allow only 100 messages
- B) Chat models have a limited context window
- C) HumanMessage cannot be repeated
- D) AIMessage stops after 10 responses

 Your answer?

 **Correct — B) Chat models have a limited context window**

One-line explanation: As conversation history grows, more tokens are sent, and eventually the model's context-window limit becomes important.

Learning conclusion: More history = more context/tokens, so history must be managed carefully.

MCQ 14 of 25

Which of the following is a practical way to manage very long conversation history?

- A) Always send every old message forever
- B) Keep recent messages and summarise older history
- C) Remove all HumanMessages
- D) Use only AIMessage

 Your answer?

✅ **Correct — B) Keep recent messages and summarise older history**

One-line explanation: For long conversations, applications may keep recent messages, summarise older history, retrieve only relevant past information, or store state externally.

Learning conclusion: Good history management means keeping useful context, not blindly sending everything.

MCQ 15 of 25

What is the main difference between a **normal text-oriented LLM flow** and a **chat model flow**?

- A) Normal LLM uses only Python, chat model uses Java
- B) Normal flow is mainly Text → Model → Text, while chat flow uses Messages → Chat Model → AIMessage
- C) Chat models cannot accept text
- D) Normal LLM always remembers previous calls

👉 **Your answer?**

✅ **Correct — B) Normal flow is mainly Text → Model → Text, while chat flow uses Messages → Chat Model → AIMessage**

One-line explanation: A normal model flow is mainly text-based, while a chat model uses structured messages with roles and returns an AIMessage.

Learning conclusion: Normal LLM = text flow; Chat Model = role-based message flow.

MCQ 16 of 25

Which message can also be included in the input history for the next model call?

- A) Only HumanMessage**
- B) Only SystemMessage**
- C) AIMessage**
- D) None of these**

 **Your answer?**

 **Correct — C) AIMessage**

One-line explanation: An AIMessage returned by the model can be stored in conversation history and sent back in the next call along with other messages.

Learning conclusion: AIMessage is not only output; it can also become part of future conversation context.

MCQ 17 of 25

What is the best reason for using SystemMessage in a conversation?

- A) To store the API key**
- B) To guide the model's behaviour, style, or context**

- C) To represent the user's latest question
- D) To permanently save chat history

👉 Your answer?

✅ **Correct — B) To guide the model's behaviour, style, or context**

One-line explanation: SystemMessage gives high-level instructions such as how the model should respond, what role it should follow, or what style it should use.

Learning conclusion: SystemMessage controls the direction and behaviour of the conversation.

MCQ 18 of 25

In a multi-turn chatbot, why do we append the model's response back into the messages list?

- A) To convert it into a SystemMessage
- B) To preserve the AI response as part of conversation history
- C) To reduce token usage automatically
- D) To stop the chatbot loop

👉 Your answer?

✅ **Correct — B) To preserve the AI response as part of conversation history**

One-line explanation: After `llm.invoke(messages)`, the returned `AIMessage` is appended to the list so future questions can use that response as context.

Learning conclusion: Store both `HumanMessage` and `AIMessage` to build proper multi-turn conversation history.

MCQ 19 of 25

What does the following line do?

```
response = llm.invoke(messages)
```

- A) Deletes the message history
- B) Sends the complete message sequence to the chat model and gets one response
- C) Converts all messages into strings
- D) Saves the conversation permanently

 **Your answer?**

 **Correct — B) Sends the complete message sequence to the chat model and gets one response**

One-line explanation: `llm.invoke(messages)` sends the current ordered message list to the chat model and returns one new `AIMessage`.

Learning conclusion: `invoke()` = give input to the model and get one response.

MCQ 20 of 25

In the simple chatbot program, what is the main purpose of:

`messages.append(response)`

- A) To print the response
- B) To save the AI response into conversation history
- C) To convert AIMessage into HumanMessage
- D) To close the model connection

 **Your answer?**

 **Correct — B) To save the AI response into conversation history**

One-line explanation: `messages.append(response)` stores the returned AIMessage in the history so later questions can use it as context.

Learning conclusion: User message + AI response should both be stored to continue the conversation properly.

MCQ 21 of 25

What does this expression give?

`type(message).__name__`

- A) The message content
- B) The class name such as SystemMessage, HumanMessage, or AIMessage

- C) The API key name
- D) The model response time

👉 Your answer?

✅ **Correct — B) The class name such as `SystemMessage`, `HumanMessage`, or `AIMessage`**

One-line explanation: `type(message).__name__` gives the message object's class name, while `message.content` gives the actual text content.

Learning conclusion: `type(message).__name__ = message type`; `message.content = message text`.

MCQ 22 of 25

In the simple interactive chatbot, what does this condition do?

```
if question.lower() == "exit":
```

```
    break
```

- A) Restarts the chatbot
- B) Clears all message history
- C) Stops the chatbot loop when the user types exit
- D) Sends exit to the model

👉 Your answer?

✅ **Correct — C) Stops the chatbot loop when the user types exit**

One-line explanation: break immediately ends the while True loop when the entered text becomes "exit" after converting it to lowercase.

Learning conclusion: break is used to stop the running chatbot loop.

MCQ 23 of 25

Which statement is correct about AIMessage?

- A) It represents the model itself
- B) It represents a message produced by the AI/model side
- C) It is always created manually
- D) It cannot be stored in conversation history

 **Your answer?**

 **Correct — B) It represents a message produced by the AI/model side**

One-line explanation: AIMessage represents the response/message coming from the AI side of the conversation, not the model itself.

Learning conclusion: ChatOpenAI = model interface;
AIMessage = message returned by the AI.

MCQ 24 of 25

Which statement about **conversation history** is correct?

- A) The model always remembers previous calls automatically
- B) The Python application can maintain previous messages and send them again as context
- C) Only SystemMessage should be stored
- D) Conversation history never affects follow-up questions

👉 Your answer?

✅ **Correct — B) The Python application can maintain previous messages and send them again as context**

One-line explanation: The simple chatbot works because the Python application stores previous Human and AI messages and sends that history back to the model.

Learning conclusion: Conversation history is maintained by the application and supplied again as context.

MCQ 25 of 25

Which flow correctly represents a LangChain chat application?

- A) Text → Text → Model
- B) SystemMessage/HumanMessage → Chat Model → AIMessage → .content
- C) AIMessage → HumanMessage → API Key
- D) SystemMessage → Python List → Database only

👉 Your answer?

✅ Correct — B) SystemMessage/HumanMessage → Chat Model → AIMessage → .content

One-line explanation: In the reference document, messages are sent to the chat model, the model returns an AIMessage, and .content gives the actual answer text.

Learning conclusion: Messages → Chat Model → AIMessage → .content → Final Answer

You completed all **25 MCQs**.

Final Score: 25 / 25 ✅

Excellent. Your understanding of **Messages and Chat Models** is very clear, especially:

SystemMessage, HumanMessage, AIMessage, invoke(), .content, conversation history, stateless calls, context window, and multi-turn chat flow.

